

Lab 10

Spring WebFlux & WebClient

1. Reactive Programming vs Blocking — ต่างกันอย่างไร
 - Blocking: เมื่อโปรแกรมรอผลลัพธ์ จะหยุด Thread ไว้จนกว่าจะทำงานเสร็จ ทำให้ใช้ทรัพยากรมากเมื่อมีหลาย Request
 - Reactive Programming: ไม่ต้องรอผลลัพธ์แบบหยุด Thread แต่ใช้การทำงานแบบ Asynchronous และจัดการข้อมูลที่เข้ามาเมื่อพร้อม ทำให้รองรับงานหลายรายการได้ดี
2. Mono vs Flux — ใช้กรณีไหน และทำไม
 - Mono<T> ใช้เมื่อคาดว่าจะมีข้อมูล 0 หรือ 1 ค่า
เช่น findById() หรือ getDiscountedPrice()
 - Flux<T> ใช้เมื่อคาดว่าจะมีข้อมูล 0 ถึงหลายค่า
เช่น findAll() หรือ findByCategory()
3. WebClient — อธิบาย method chain .get(), .uri(), .retrieve(), .bodyToMono/Flux()
 - .get() กำหนด HTTP Method เป็น GET
 - .uri() กำหนด URL ของ API
 - .retrieve() ส่ง Request และเตรียมรับ Response
 - .bodyToMono(Product.class) แปลง Response Body เป็น Mono<Product>
 - .bodyToFlux(Product.class) แปลง Response Body ที่มีหลายรายการเป็น Flux<Product>

4. Code ของ Repository, Service, Controller พร้อม comment อธิบาย

Repository

```
public Mono<Product> findById(String id) {  
    // TODO: เติม code ตรงนี้  
    // ค้นหา Product จาก id  
    Product product = store.get(id);  
    // ถ้าไม่พบข้อมูล ให้คืน Mono.empty()  
    if (product == null) {  
        return Mono.empty();  
    }  
    // ถ้าพบ ให้คืน Product ในรูปแบบ Mono  
    return Mono.just(product);  
}  
  
public Flux<Product> findAll() {  
    // TODO: เติม code ตรงนี้  
    // แปลง Product ทั้งหมดใน store เป็น Flux  
    return Flux.fromIterable(store.values());  
}  
  
// — 3. บันทึก Product —————  
/**  
 * TODO: บันทึก product ลง store แล้วคืน Mono<Product>  
 *  
 * Hint: store.put(product.getId(), product)  
 *       แล้วใช้ Mono.just(product) คืนค่า  
 */  
public Mono<Product> save(Product product) {  
    // TODO: เติม code ตรงนี้  
    // บันทึก Product ลงใน store  
    store.put(product.getId(), product);  
    return Mono.just(product);  
}
```

```
public Mono<Void> deleteById(String id) {  
    // TODO: เติม code ตรงนี้  
    // ลบ Product ตาม id  
    store.remove(id);  
    return Mono.empty();  
}  
  
// — 5. กรองตาม category —  
/**  
 * TODO: คีน Flux<Product> ที่ category ตรงกัน  
 *  
 * Hint: findAll()  
 *       .filter(p -> p.getCategory().equalsIgnoreCase(category))  
 */  
public Flux<Product> findByCategory(String category) {  
    // TODO: เติม code ตรงนี้  
    // กรอง Product ตาม category  
    return findAll()  
        .filter(p -> p.getCategory().equalsIgnoreCase(category));  
}
```

Service

```
public Mono<Product> getById(String id) {  
    // TODO: เติม code ตรงนี้  
    // เรียก Repository และแจ้ง Error หากไม่พบข้อมูล  
    return repository.findById(id)  
        .switchIfEmpty(  
            Mono.error(  
                new RuntimeException("Product not found: " + id)  
            )  
        );  
}  
  
// — 2. ดึง Product ทั้งหมด —————  
/**  
 * TODO: เรียก repository.findAll() แล้วคืนผล  
 */  
public Flux<Product> getAll() {  
    // TODO: เติม code ตรงนี้  
    // เรียกข้อมูล Product ทั้งหมด  
    return repository.findAll();  
}  
  
// — 3. บันทึก Product —————  
/**  
 * TODO: เรียก repository.save(product) แล้วคืนผล  
 *  
 * เพิ่มเติม: ถ้า product.getId() เป็น null ให้ generate id ใหม่  
 * Hint: java.util.UUID.randomUUID().toString()  
 */  
public Mono<Product> save(Product product) {  
    // TODO: เติม code ตรงนี้  
    // สร้าง ID อัตโนมัติถ้ายังไม่มี ID  
    if (product.getId() == null) {  
        product.setId(UUID.randomUUID().toString());  
    }  
    return repository.save(product);  
}
```

```

// — 4. ลบ Product —————
/**
 * TODO: เรียก repository.deleteById(id) แล้วคืนผล
 */
public Mono<Void> delete(String id) {
    // TODO: เติม code ตรงนี้
    // ลบ Product ผ่าน Repository
    return repository.deleteById(id);
}

// — 5. กรองตาม category —————
/**
 * TODO: เรียก repository.findByCategory(category) แล้วคืนผล
 */
public Flux<Product> getCategory(String category) {
    // TODO: เติม code ตรงนี้
    // ค้นหา Product ตาม category
    return repository.findByCategory(category);
}

// — 6. คำนวณราคาหลังส่วนลด —————
/**
 * TODO: หา Product จาก id แล้วคืน discountedPrice
 *
 * Hint: getById(id)
 *         .map(p -> p.getDiscountedPrice())
 */
public Mono<Double> getDiscountedPrice(String id) {
    // TODO: เติม code ตรงนี้
    // ดึง Product แล้วคำนวณราคาหลังส่วนลด
    return getById(id)
        .map(p -> p.getDiscountedPrice());
}

```

Controller

```
/**
 * @GetMapping("/{id}")
 * public Mono<Product> getById(@PathVariable String id) {
 *     // รับ ID แล้วส่งต่อให้ Service
 *     return service.getById(id);
 * }
 *
 * // =====
 * // ✖ TODO: เติม method body ด้านล่างนี้
 * // =====
 *
 * /**
 * * GET /products
 * * TODO: คืน Flux<Product> ทุกรายการ
 * *
 * * Hint: เรียก service.getAll()
 * * ทดสอบ: GET http://localhost:8080/products
 * */
 * @GetMapping
 * public Flux<Product> getAll() {
 *     // TODO: เติม code ตรงนี้
 *     // ดึง Product ทั้งหมด
 *     return service.getAll();
 * }
 *
 * /**
 * * @PostMapping
 * * public Mono<Product> save(@RequestBody Product product) {
 * *     // TODO: เติม code ตรงนี้
 * *     // รับ Product จาก Request แล้วบันทึก
 * *     return service.save(product);
 * * }
 *
 * /**
 * * DELETE /products/{id}
 * * TODO: ลบ Product และคืน Mono<Void>
 * *
 * * Hint: เรียก service.delete(id)
 * * ทดสอบ: DELETE http://localhost:8080/products/1
 * */
 * @DeleteMapping("/{id}")
 * public Mono<Void> delete(@PathVariable String id) {
 *     // TODO: เติม code ตรงนี้
 *     // ลบ Product ตาม ID
 *     return service.delete(id);
 * }
 */
```

```

/**
 * GET /products/category/{category}
 * TODO: คืน Flux<Product> ที่กรองตาม category
 *
 * Hint: เรียก service.getByCategory(category)
 * ทดสอบ: GET http://localhost:8080/products/category/Electronics
 */
@GetMapping("/category/{category}")
public Flux<Product> getCategory(@PathVariable String category) {
    // TODO: เติม code ตรงนี้
    // ค้นหา Product ตาม Category
    return service.getByCategory(category);
}

/**
 * GET /products/{id}/price
 * TODO: คืน Mono<Double> ราคาหลังส่วนลด
 *
 * Hint: เรียก service.getDiscountedPrice(id)
 * ทดสอบ: GET http://localhost:8080/products/1/price
 */
@GetMapping("/{id}/price")
public Mono<Double> getDiscountedPrice(@PathVariable String id) {
    // TODO: เติม code ตรงนี้
    // คืนราคาหลังส่วนลด
    return service.getDiscountedPrice(id);
}

```

4. ผลการทดสอบ

4.1 GET Product

Lab9 > <http://localhost:8080/products>

GET <http://localhost:8080/products/1>

Docs Params Authorization Headers 6 Body Scripts Settings

Body Cookies Headers 2 Test Results 200 OK

{ } JSON Preview Visualize

```
1 {
2   "id": "1",
3   "name": "iPhone 15 Pro (673380033-9 SEC 1)",
4   "category": "Electronics",
5   "brand": "Apple",
6   "stock": 50,
7   "price": 39900.0,
8   "discountType": "MEMBER",
9   "discountedPrice": 35910.0
10 }
```

4.2 GET All Products

 Lab9 > <http://localhost:8080/products>

GET

<http://localhost:8080/products>

Docs

Params

Authorization

Headers 6

Body

Scripts

Settings

Body

Cookies

Headers 2

Test Results



{ } JSON

Preview

Visualize

```
1  [
2    {
3      "id": "1",
4      "name": "iPhone 15 Pro (673380033-9 SEC 1)",
5      "category": "Electronics",
6      "brand": "Apple",
7      "stock": 50,
8      "price": 39900.0,
9      "discountType": "MEMBER",
10     "discountedPrice": 35910.0
11   },
12   {
13     "id": "2",
14     "name": "MacBook Air M3",
15     "category": "Electronics",
16     "brand": "Apple",
17     "stock": 20,
18     "price": 49900.0,
19     "discountType": "NONE",
20     "discountedPrice": 49900.0
21   },
22   {
23     "id": "3",
24     "name": "Samsung Galaxy S24",
25     "category": "Electronics",
26     "brand": "Samsung",
27     "stock": 30,
28     "price": 29900.0,
```

4.3 POST Product\

POST

http://localhost:8080/products

Docs

Params

Authorization

Headers 8

Body

Scripts

Settings

none

form-data

x-www-form-urlencoded

raw

binary

GraphQL

JSON

```
1 {
2   "name": "Test Product",
3   "category": "Electronics",
4   "brand": "Test Brand",
5   "stock": 10,
6   "price": 1000.0,
7   "discountType": "MEMBER"
8 }
```

Body

Cookies

Headers 2

Test Results

200 OK

{ } JSON

Preview

Visualize

```
1 {
2   "id": "179adb71-dca5-4d0d-b61e-df1015b5494a",
3   "name": "Test Product",
4   "category": "Electronics",
5   "brand": "Test Brand",
6   "stock": 10,
7   "price": 1000.0,
8   "discountType": "MEMBER",
9   "discountedPrice": 900.0
10 }
```

จะได้ ID : 179adb71-dca5-4d0d-b61e-df1015b5494a

4.4 DELETE Product

DELETE

http://localhost:8080/products/179adb71-dca5-4d0d-b61e-df1015b5494a

Docs

Params

Authorization

Headers 8

Body

Scripts

Settings

Document this request...

Write with AI

Body

Cookies

Headers 1

Test Results

200 OK

Raw

Preview

Visualize

```
1
```

4.5 GET Category

GET

http://localhost:8080/products/category/Electronics

Docs

Params

Authorization

Headers 8

Body

Scripts

Settings

Body

Cookies

Headers 2

Test Results

200 OK

JSON

Preview

Visualize

```
1  [
2    {
3      "id": "1",
4      "name": "iPhone 15 Pro (673380033-9 SEC 1)",
5      "category": "Electronics",
6      "brand": "Apple",
7      "stock": 50,
8      "price": 39900.0,
9      "discountType": "MEMBER",
10     "discountedPrice": 35910.0
11   },
12   {
13     "id": "2",
14     "name": "MacBook Air M3",
15     "category": "Electronics",
16     "brand": "Apple",
17     "stock": 20,
18     "price": 49900.0,
19     "discountType": "NONE",
20     "discountedPrice": 49900.0
21   },
22   {
23     "id": "3",
24     "name": "Samsung Galaxy S24",
25     "category": "Electronics",
26     "brand": "Samsung",
27     "stock": 30,
28     "price": 29900.0,
29     "discountType": "SEASONAL",
30     "discountedPrice": 23920.0
31   }
32 ]
```

4.6 GET Discounted Price

GET

http://localhost:8080/products/1/price

Docs

Params

Authorization

Headers 8

Body

Scripts

Settings

Document this request... Write with AI

Body

Cookies

Headers 2

Test Results

200 OK

JSON

Preview

Visualize

```
1  35910.0
```